TÀI LIỆU HỌC VÀ CHUẨN BỊ THUYẾT TRÌNH

Đào Văn Tâm (B25CHHT112) · Nhóm 8 · Tìm kiếm và Truy xuất thông tin · Đề tài LinCIR

1. Vai trò của bạn

Chuyển đổi mã nguồn LinCIR sang môi trường mới, kiểm chứng tương đương với mô hình gốc; vận hành thực nghiệm trên Kaggle: lập lịch chạy song song trong hạn mức GPU, khắc phục nghẽn nạp dữ liệu.

Slide	Nội dung	Thời gian
1	Bìa, giới thiệu nhóm	0:20
9	Dựng mã nguồn	1:00
13	Thiết lập thực nghiệm	0:45

2. Bức tranh chung của đề tài (cả nhóm cần nắm)

Bài toán. Truy xuất ảnh kết hợp (Composed Image Retrieval – CIR): truy vấn gồm một ảnh tham chiếu và một câu mô tả thay đổi ("chiếc váy này nhưng màu bạc, tay ngắn hơn"); hệ thống trả về ảnh thỏa cả hai. Dữ liệu bộ ba ⟨ảnh, câu, ảnh đích⟩ đắt và nhỏ, nên hướng **zero-shot** huấn luyện không cần bộ ba.

Bài báo. LinCIR (Gu và cộng sự, CVPR 2024) học một mạng chiếu φ (MLP 3 lớp) biến một vector thành "từ giả" [$] trong không gian token của CLIP, **chỉ bằng văn bản**: che các từ khóa của câu bằng [$] = φ(vector của câu + nhiễu) và ép câu đã che có cùng vector với câu gốc (Self-Masking Projection, mất mát MSE). Khi truy vấn, thay vector câu bằng vector ảnh: "a photo of [$] that {câu điều chỉnh}" → tìm ảnh theo cosine.

Nhóm đã làm. (1) Chuyển mã nguồn chính thức sang Python 3.13 / transformers 5.x, kiểm chứng sai lệch 0 so với mô hình gốc. (2) Tái hiện: checkpoint chính thức đạt FashionIQ R@10 = 26,35 / R@50 = 46,12 (bài báo 26,28 / 46,49). (3) Hai cải tiến: **A** – thêm mất mát đối sánh InfoNCE; **B** – nhiễu có hướng theo modality gap. (4) Huấn luyện baseline, A, B, A+B trên Kaggle (2 × T4), mỗi biến thể 3 seed, 5.000 bước; đánh giá FashionIQ val + CIRCO val; kiểm định thống kê theo truy vấn.

Kết quả chính. B (nhiễu có hướng) tốt hơn baseline có ý nghĩa thống kê: FashionIQ R@10 +0,60 ($p = 0{,}006$), R@50 +1,27 ($p < 0{,}001$), CIRCO mAP@25/50 tăng có ý nghĩa; B còn vượt checkpoint chính thức 20.000 bước. **A (InfoNCE) không tốt hơn** trên FashionIQ và **kém rõ trên CIRCO** (mAP@5 −2,45, KTC 95% nằm dưới 0); A+B tương tự A. Giải thích: độ nhất quán của φ giữa ảnh và văn bản (cosine) B 0,84 > baseline 0,80 > A 0,64 — cùng thứ tự với CIRCO. Kết luận: H2 được ủng hộ, H1 và H3 bị bác bỏ.

Hạng mục	Số liệu / cấu hình
Backbone	CLIP ViT-L/14 (đóng băng); chỉ huấn luyện φ
Huấn luyện	AdamW, lr 1e-4, wd 0,01, batch 512, dropout 0,5, 5.000 bước (bài báo 20.000)
Dữ liệu train	Chỉ văn bản: chú thích CC3M + 2,47 triệu prompt Stable Diffusion +

	prompt Midjourney
Đánh giá	FashionIQ val (6.016 truy vấn, 3 nhóm, R@10/R@50); CIRCO val (220 truy vấn, 123.403 ảnh, mAP@K)
Tái hiện	Checkpoint chính thức: FIQ R@10 26,35 vs 26,28; R@50 46,12 vs 46,49 (lệch lớn nhất 0,77 điểm)
Cải tiến A	$L = MSE + 0{,}5 \cdot InfoNCE$ ($\tau = 0{,}07$, 511 mẫu âm trong batch)
Cải tiến B	$n = 0{,}5 \cdot \Delta + u \cdot \varepsilon$; Δ = tâm ảnh − tâm văn bản trên 5.000 cặp COCO ($\|\Delta\| = 13{,}87$)
Giao thức	3 seed (12345, 2024, 7), checkpoint bước cuối, paired bootstrap 10.000 lần + Wilcoxon, kết luận khi $p < 0{,}05$
Tài nguyên	Kaggle 2 × T4 16 GB, 30 giờ GPU/tuần, giới hạn 12 giờ/phiên; 1,45 s/bước, 2,0 giờ/run
Kết quả (TB 3 seed)	FIQ R@10: baseline 25,90 · A 25,54 · B 26,50 · A+B 25,44; CIRCO mAP@5: 10,47 · 8,02 · 10,94 · 7,81

3. Trình bày rõ ràng, dễ hiểu (đọc trước khi tập)

Mẹo thuyết trình

- **Mở đầu bằng mạch chính trong 3 câu:** (1) Truy xuất ảnh kết hợp cần dữ liệu có nhãn rất tốn kém. (2) LinCIR huấn luyện chỉ bằng văn bản nên không cần dữ liệu đó. (3) Nhóm tái hiện LinCIR, thử hai cải tiến, chứng minh bằng kiểm định rằng một cải tiến tốt hơn và giải thích vì sao cải tiến còn lại kém hơn.
- **Mỗi slide theo trình tự:** nêu ý chính (tiêu đề) → nêu một con số → giải thích con số đó có nghĩa gì → nối sang slide sau. Không đọc toàn bộ bảng; chỉ nêu dòng hoặc cột cần thiết.
- **Diễn giải con số ngay khi nêu:** R@10 = 26 nghĩa là trong 26% số truy vấn, ảnh đúng nằm trong 10 kết quả đầu tiên; tăng 0,60 điểm R@10 tương đương thêm khoảng 6 truy vấn tìm đúng trên mỗi 1.000 truy vấn.
- **Thuật ngữ:** nêu tên tiếng Việt kèm tên tiếng Anh và định nghĩa một câu ở lần xuất hiện đầu tiên; các lần sau dùng đúng một tên đó.
- **Khi nói, làm tròn hợp lý** ("khoảng 26%", "giảm khoảng 23%"); số chính xác đã có trên slide và trong báo cáo.
- **Ngắt nghỉ sau kết luận chính** (slide 14) trước khi chuyển sang phần giải thích nguyên nhân.
- **Trả lời câu hỏi:** câu đầu nêu thẳng kết luận, câu sau nêu căn cứ (số liệu, mục trong báo cáo).

Slide của bạn: ý chính và con số cần nêu

Slide	Ý chính nói thành lời	Một con số và ý nghĩa
1	Nhóm tái hiện phương pháp LinCIR, đề xuất hai cải tiến và kiểm chứng bằng kiểm định thống kê.	—
9	Mã nguồn gốc không chạy với thư viện hiện tại. Nhóm viết lại phần mã hóa văn bản, sửa 4 lỗi và kiểm chứng	Sai lệch 0,00 so với CLIP gốc

	kết quả trùng khớp với mô hình gốc.	
13	Mọi biến thể dùng cùng dữ liệu, cùng 5.000 bước, cùng siêu tham số và chạy 3 seed; chỉ khác đúng thành phần cải tiến. Nhóm khắc phục nghẽn nạp dữ liệu để kịp chạy trong hạn mức GPU.	Tốc độ tăng 3,1 lần: 2,0 giờ mỗi lần chạy thay vì 6,2 giờ

Thuật ngữ: định nghĩa ngắn

Thuật ngữ	Định nghĩa	Ví dụ trong đề tài
Truy xuất ảnh kết hợp (CIR)	Tìm ảnh bằng truy vấn gồm một ảnh tham chiếu và một câu mô tả thay đổi	Ảnh váy hoa xanh + "màu bạc, tay ngắn hơn" → tìm ảnh váy bạc tay ngắn
Zero-shot	Mô hình không được huấn luyện trên dữ liệu có nhãn của bài toán đích, được đánh giá trực tiếp	LinCIR không dùng bộ ba nào của FashionIQ hay CIRCO khi huấn luyện
Bộ ba có nhãn	Ảnh tham chiếu – câu điều chỉnh – ảnh đích, do người gán thủ công	FashionIQ có khoảng 46,6 nghìn bộ ba huấn luyện
CLIP	Mô hình gồm bộ mã hóa ảnh và bộ mã hóa văn bản, đưa ảnh và câu mô tả tương ứng về gần nhau	Nhóm dùng CLIP ViT-L/14, giữ nguyên không huấn luyện lại
Vector nhúng	Dãy số (768 chiều với ViT-L) biểu diễn nội dung của một ảnh hoặc một câu	Mỗi ảnh trong kho được mã hóa thành một vector
Độ tương đồng cosine	Mức độ cùng hướng của hai vector, càng gần 1 càng giống	Kết quả truy xuất được xếp theo cosine giữa truy vấn và từng ảnh
Token giả [$]	Một vector đặt vào vị trí của một từ trong câu, đại diện cho ảnh tham chiếu	Câu truy vấn: "a photo of [$] that is silver with shorter sleeves"
Mạng chiếu φ	Mạng nơ-ron nhỏ (3 lớp) biến vector thành token giả; là thành phần duy nhất được huấn luyện	Cả 4 biến thể của nhóm chỉ khác nhau ở cách huấn luyện φ
Self-Masking Projection (SMP)	Che các cụm tính từ – danh từ của câu bằng [$] và huấn luyện φ để câu đã che giữ nguyên vector của câu gốc	"gray cat sleeps on a pillow" → "[$] sleeps on [$]"
Khoảng cách ảnh – văn bản (modality gap)	Vector ảnh và vector văn bản của CLIP nằm ở hai vùng tách biệt	Ảnh và chú thích của chính nó chỉ có cosine trung bình 0,26 (5.000 cặp COCO)
Nhiễu khi huấn luyện	Thành phần ngẫu nhiên cộng vào vector văn bản để φ chịu được khác biệt giữa ảnh và văn bản	LinCIR dùng $n = u \cdot \varepsilon$, $u \sim U(0,1)$, $\varepsilon \sim N(0, I)$
Cải tiến B – nhiễu có hướng	Cộng thêm vector Δ (tâm vector ảnh trừ tâm vector văn bản) vào nhiễu, đẩy đầu vào của φ về phía vùng ảnh	$n = 0{,}5 \cdot \Delta + u \cdot \varepsilon$, Δ tính trên 5.000 cặp COCO
Cải tiến A – InfoNCE	Hàm mất mát đối sánh: mỗi câu phải gần chính nó hơn 511 câu khác trong cùng batch	$L = MSE + 0{,}5 \cdot InfoNCE$

Baseline	LinCIR gốc do nhóm tự huấn luyện, cùng cấu hình với các biến thể, dùng làm mốc so sánh	5.000 bước, 3 seed
Checkpoint	Trọng số của mô hình được lưu tại một thời điểm huấn luyện	Checkpoint chính thức của tác giả huấn luyện 20.000 bước
Bước huấn luyện	Một lần cập nhật trọng số trên một batch 512 câu	5.000 bước mất khoảng 2 giờ trên GPU T4
Seed	Giá trị khởi tạo bộ sinh số ngẫu nhiên; khác seed thì thứ tự dữ liệu và nhiễu khác nhau	Mỗi biến thể chạy với 3 seed: 12345, 2024, 7
Độ lệch chuẩn (±)	Mức dao động của kết quả giữa các lần chạy khác seed	B: R@10 = 26,50 ± 0,04 — rất ổn định
R@10, R@50 (Recall@K)	Tỷ lệ truy vấn có ảnh đích nằm trong K kết quả đầu	FashionIQ chỉ có một ảnh đúng cho mỗi truy vấn
mAP@5	Trung bình độ chính xác trên K = 5 kết quả đầu, dùng khi mỗi truy vấn có nhiều ảnh đúng	CIRCO có trung bình 4,16 ảnh đúng mỗi truy vấn
p-value	Xác suất quan sát được mức chênh lệch như vậy nếu thực tế hai phương pháp ngang nhau	B so với baseline, FashionIQ R@10: p = 0,006
Khoảng tin cậy 95% (KTC)	Khoảng giá trị của mức chênh lệch thật với độ tin cậy 95%; nếu toàn bộ khoảng lớn hơn 0 thì cải tiến tốt hơn	B, R@50: Δ = +1,27, KTC [+0,81; +1,73]
Paired bootstrap	Lấy mẫu lại các truy vấn 10.000 lần, tính lại mức chênh mỗi lần để ước lượng p-value và KTC	So từng truy vấn của B với cùng truy vấn đó của baseline
Âm tính giả	Ảnh thực tế thỏa truy vấn nhưng không được gán nhãn là đáp án	Lý do bài báo khuyến nghị dùng mAP và K lớn thay vì R@1
Ablation	So sánh các biến thể bật/tắt từng thành phần để tách đóng góp của mỗi thành phần	Baseline, A, B, A+B

4. Kiến thức và nội dung công việc

Bộ mã hóa văn bản của CLIP

- Câu → tokenizer BPE → chuỗi mã token (tối đa 77, bắt đầu 49406, kết thúc EOS 49407) → lớp nhúng token + nhúng vị trí → 12 khối Transformer có **mặt nạ nhân quả** (mỗi token chỉ nhìn các token phía trước) → LayerNorm cuối → lấy vector tại vị trí EOS → phép chiếu tuyến tính → vector văn bản z_c (768 chiều với ViT-L).
- Vị trí EOS được tìm bằng argmax của mã token vì 49407 là mã lớn nhất trong câu. Do mặt nạ nhân quả, các token đệm phía sau EOS không ảnh hưởng tới kết quả.
- LinCIR chèn token giả bằng cách: ở mọi vị trí có mã 259 (ký tự "$"), thay vector nhúng token bằng $\hat{e} = \varphi(\cdot)$ trước khi đi qua các khối Transformer.

Vì sao phải viết lại

- Mã gốc gọi hàm nội bộ text_model.encoder(..., causal_attention_mask=...) của transformers 4.34; ở transformers 5.x tham số này không còn. Nhóm viết lại lượt truyền xuôi từ các mô-đun con có tên ổn định qua các phiên bản: layer_norm1, q/k/v/out_proj, layer_norm2, mlp, với scaled_dot_product_attention(is_causal=True).
- Gói clip của OpenAI (chỉ dùng clip.tokenize) khó cài trên Python ≥ 3.13 → thay bằng tokenizer CLIP của HuggingFace (cùng bảng từ vựng BPE, "$" vẫn là mã 259).

Kiểm chứng tương đương (cách chứng minh "đúng")

- So vector của encoder viết lại với CLIPTextModelWithProjection gốc trên cùng câu (không có token giả): sai lệch tuyệt đối lớn nhất **0,00** trên ViT-B/32 (Mac) và ViT-L/14 (GPU Kaggle) — scripts/check_encoder_equivalence.py.
- Token "$" = 259 ở cả tokenizer đánh giá và tokenizer huấn luyện ("[$]" thêm vào tokenizer SDXL có mã 49408, được đổi thành 259).
- Checkpoint chính thức nạp khớp hoàn toàn (strict) và tái hiện FashionIQ lệch ≤ 0,77 điểm so với bài báo; chạy đánh giá hai lần cho thứ hạng từng truy vấn giống hệt.
- Các lỗi đã sửa trong mã gốc: gõ nhầm arg → args khi cắt gradient; truyền biến chưa định nghĩa khi đánh giá CIRCO; ép .half() gây lỗi trên CPU/MPS; ghi tham số list/None vào TensorBoard làm crash trên Kaggle.

Vận hành trên Kaggle

- Mỗi phiên tối đa 12 giờ, hạn mức 30 giờ GPU/tuần **tính theo giờ phiên** (đã kiểm chứng: 2:25 → 8:43 sau phiên 6,3 giờ dù dùng 2 GPU).
- Chạy 2 run song song, mỗi run 1 T4 với batch 512 (bộ nhớ đỉnh 11,4/15 GB). Không dùng 2 GPU cho 1 run vì khi đó InfoNCE chỉ thấy 256 mẫu âm mỗi GPU → khác cấu hình giữa các biến thể.
- Bộ lập lịch tự **ước lượng thời gian** run (học từ run đã xong) và không bắt đầu run mới nếu không kịp trước 11,3 giờ → không bao giờ bị Kaggle cắt giữa chừng và mất output. Mỗi phiên tự "mang theo" checkpoint, dữ liệu đánh giá của phiên trước (carry-forward).

Khắc phục nghẽn nạp dữ liệu

- Triệu chứng: chạy song song 4,5 giây/bước (dự kiến ~2,4) → 6,2 giờ/run. Đo trên caption thật: spaCy tốn 4,7 ms/câu, tokenizer 0,1 ms → GPU chờ CPU (Kaggle chỉ 4 vCPU cho 2 run).
- Giải pháp: spaCy xử lý theo lô (nlp.pipe) và bỏ thành phần NER, lemmatizer (không ảnh hưởng nhãn từ loại) → 1,51 ms/câu (nhanh ×3,1).
- Bằng chứng không đổi kết quả: từ khóa giống hệt trên 3.000 câu thật; DataLoader cùng seed cho các batch giống hệt từng token với mã cũ. Phương án bỏ cả parser bị loại vì làm khác kết quả ở 48/1.536 câu.

- Tốc độ sau tối ưu: **1,45 giây/bước khi 2 run song song** (2,0 giờ/run, trước đó 6,2 giờ) → 8 run trong một phiên thay vì 2.

Tệp liên quan trong dự án

- code/encode_with_pseudo_tokens.py, code/compat.py — encoder viết lại, tokenizer thay thế
- scripts/check_encoder_equivalence.py — kiểm chứng sai lệch 0
- code/loader.py (CaptionDataset.collate) — spaCy theo lô
- scripts/run_experiment.py, scripts/kaggle_jobs.py, notebooks/kaggle_lincir.ipynb — chạy thí nghiệm, lập lịch, carry-forward
- docs/decisions.md — lý do của từng quyết định

5. Kịch bản trình bày chi tiết

Slide	Ý chính cần nói
1	Giới thiệu đề tài, môn học, giảng viên và 5 thành viên; nêu một câu mục tiêu: tái hiện và cải tiến LinCIR, chứng minh bằng kiểm định thống kê.
9	Mã gốc viết cho transformers 4.34, không chạy trên môi trường hiện tại. Nhóm viết lại encoder từ các mô-đun con, thay tokenizer và sửa 4 lỗi. Bằng chứng: sai lệch 0 so với CLIP gốc, checkpoint chính thức nạp khớp, kết quả khớp bài báo (chuyển sang Tùng). Nêu 3 điểm khác giữa bài báo và mã nguồn để giải thích vì sao baseline theo mã.
13	Thiết lập: Kaggle 2 × T4; mỗi biến thể 3 seed, 5.000 bước, cùng siêu tham số, checkpoint bước cuối; FashionIQ + CIRCO. Nêu hai vấn đề vận hành và cách giải: giới hạn 12 giờ/30 giờ (lập lịch có ước lượng thời gian), nghẽn CPU (spaCy theo lô, ×3,1, kết quả giống hệt).

6. Câu hỏi phản biện dự kiến

H1. Làm sao chắc encoder viết lại cho kết quả giống bản gốc?

→ So trực tiếp vector đầu ra với CLIPTextModelWithProjection gốc trên cùng đầu vào: sai lệch lớn nhất 0,00 trên cả ViT-B/32 và ViT-L/14. Thêm vào đó, checkpoint chính thức tái hiện khớp bài báo (lệch ≤ 0,77 điểm) và đánh giá lặp lại cho thứ hạng từng truy vấn giống hệt.

H2. Vì sao lấy vector tại EOS bằng argmax?

→ EOS có mã 49407, lớn nhất trong bảng từ, nên argmax của dãy mã chính là vị trí EOS. Mặt nạ nhân quả làm các token đệm sau EOS không ảnh hưởng.

H3. Tại sao chỉ 5.000 bước, có làm sai lệch kết luận không?

→ 20.000 bước trên T4 ≈ 13 giờ/run, vượt giới hạn phiên 12 giờ và hạn mức tuần. Mọi biến thể cùng 5.000 bước nên so sánh vẫn công bằng; số tuyệt đối có thể thấp hơn checkpoint 20.000 bước. Đường học (Trung trình bày): R@10 chỉ tăng 0,25 điểm từ 4.000 đến 5.000 bước; baseline 5.000 bước kém checkpoint 20.000 bước 0,45 điểm R@10 và gần bằng trên CIRCO (10,47 so với 10,50).

H4. Chạy 2 run song song có ảnh hưởng kết quả?

→ Không: mỗi run một GPU riêng, cùng cấu hình; chỉ ảnh hưởng tốc độ. Kết quả phụ thuộc seed và dữ liệu, không phụ thuộc việc chạy cùng lúc.

H5. Bỏ NER và lemmatizer có đổi dữ liệu huấn luyện không?

→ Không. Hai thành phần này không ghi nhãn từ loại mà LinCIR dùng để chọn từ khóa. Đã kiểm tra từ khóa giống hệt trên 3.000 câu thật và batch giống hệt từng token với mã cũ. Riêng parser thì có ảnh hưởng nên được giữ lại.

H6. Tại sao không dùng 2 GPU cho 1 run cho nhanh?

→ Huấn luyện phân tán chia batch 512 thành 2 × 256; InfoNCE (cải tiến A) chỉ thấy mẫu âm trên GPU của mình → khác cấu hình với baseline. Chạy 2 run độc lập giữ batch 512 cho mọi biến thể.

H7. fp16 có làm khác kết quả so với bài báo?

→ Mã gốc cũng huấn luyện fp16. Khi đánh giá nhóm dùng fp32 như mã validate gốc; tái hiện khớp bài báo.

7. Tự kiểm tra trước buổi thuyết trình

- ☐ Vẽ lại được luồng: câu → token → nhúng → (thay [$]) → Transformer nhân quả → EOS → chiếu.
- ☐ Giải thích vì sao sai lệch 0 là bằng chứng đủ mạnh, và còn những bằng chứng nào khác.
- ☐ Nói được tốc độ trước/sau tối ưu và vì sao kết quả không đổi.
- ☐ Giải thích cách tính hạn mức Kaggle và cơ chế không để phiên bị cắt.

8. Đọc thêm

- Báo cáo: mục 3.5, 5.1, 5.3; docs/decisions.md
- Bài báo LinCIR: mục 3.1, 4.1
- CLIP (Radford 2021): mục 2.3 (kiến trúc encoder văn bản)